

Sanmill als Grundlage für ein eigenes iOS-Produkt

Entscheidungsgrundlage, Wiederverwendung und Umsetzung
Stand: 30. September 2026 · Vorplanung, Version 1.0

Empfohlener Weg

Eine eigenständig gestaltete SwiftUI-App mit übernommenem Rust-Spielkern. Der vollständige zugehörige App-Quellcode wird offen veröffentlicht. Zielpreis: **einmalig 0,99 € im deutschen App Store**. Die Netzwerkfunktion folgt nach einer stabilen Offline-Version.

Was damit bereits geklärt ist

Die AGPL gestattet grundsätzlich kommerziellen Vertrieb. Für die gewünschte Wiederverwendung müssen wir die Mühle-Regeln und die KI-Suche nicht neu erfinden. Sanmill nennt zusätzlich eine App-Store-Erlaubnis. Deren Reichweite für eingebundene Fremdbeiträge bleibt ein gezielter Prüfpunkt vor der Veröffentlichung. [S1, S2, S3]

„Nativ“ bedeutet hier: SwiftUI für die gesamte Oberfläche und Apple-Integration, Rust als auf dem Gerät kompilierte Bibliothek für Regeln und Suche. Es ist dafür weder eine Flutter-Oberfläche noch ein Server für KI-Züge erforderlich. Die Swift-Anbindung schreiben wir neu. [S4, S17]

Vorgaben und Planungsannahmen

Festgelegt durch den Auftrag	Vorschlag für die Umsetzung
Open Source ist akzeptiert; Verkauf für 99 Cent.	Bezahlter Download; keine Werbung und kein Konto für das Offline-Spiel.
Layout, Aufbau, Spielmöglichkeiten und KI erhalten.	Bewährte Informationsordnung erhalten, Darstellung und Bedienung für iOS neu entwickeln.
Netzwerkfähigkeit in einem späteren Schritt.	Jetzt Zustände und Zugprotokoll vorbereiten, später Game Center oder eigenes Backend anbinden.
Erst Namensgebung und Grunddesign; danach Projektordner.	Jetzt nur Recherche und Planung. Produktname, Bundle-ID, Repository und Xcode-Projekt erst nach der Gestaltung festlegen.

Aufwand als erste Arbeitshypothese

Für die unten definierte Offline-Version: **58-91 Personentage**, mit 20 % Reserve rund **70-110 Personentage**. Bei einer Vollzeit-Entwicklung entspricht das etwa **14-22 Arbeitswochen**. Externe Rechtsprüfung, Rückmeldungen und App Review können die Kalenderzeit verlängern. Neu schätzen nach dem Integrationsprototyp. Netzwerk separat: etwa 4-7 Wochen mit Game Center oder 8-14 Wochen für einen eigenen Dienst.

Quellen: [S1 Lizenz](#) · [S2 App-Store-Erlaubnis](#) · [S4 Architektur](#)

Was wir rechtlich dürfen

Verkaufen und wiederverwenden: grundsätzlich ja

Sanmills Lizenztext erlaubt die entgeltliche Weitergabe. Die geprüften Rust-Pakete und zahlreiche Dateiköpfe nennen **AGPL-3.0-or-later**. Wir können sie verwenden, ändern und in eine native App einbinden. Für das integrierte Produkt planen wir die gesamte abgeleitete App einschließlich SwiftUI-Code unter AGPL. Eine Sprachübersetzung, ein Wrapper oder statisches Linken beseitigt diese Pflichten nicht. [S1, S3, S5]

Zu jedem ausgelieferten Stand stellen wir den vollständigen zugehörigen Quellcode mit Build-Anleitung, notwendigen Skripten, Abhängigkeiten und Änderungen bereit. Lizenz- und Urheberhinweise bleiben erhalten. Ein Link nur auf Sanmills unverändertes Repository genügt nicht. Praktisch: öffentliches Release-Archiv passend zur App-Version und ein Quellcode-Link in der App. Persönliche Signierschlüssel und Betriebsgeheimnisse werden nicht veröffentlicht. [S1]

Käufer dürfen den freigegebenen Code weiterverwenden und unter den Lizenzbedingungen weiterverbreiten, auch kostenlos. Der Preis vergütet unser fertiges App-Store-Produkt; er verschafft kein Monopol auf den Code. Reine interne Entwicklung löst nicht automatisch eine öffentliche Veröffentlichungspflicht aus; Distribution an Tester ist früh mit zu berücksichtigen. [S1]

App Store: konkrete Erlaubnis, aber kein pauschaler Freibrief

Die README enthält ausdrücklich eine zusätzliche Erlaubnis nach Abschnitt 7 für App Stores mit sonst unvereinbaren Beschränkungen, bei weiter zugänglichem AGPL-Quellcode. Der Zusatz steht in der README, nicht als eigener Anhang im geprüften Copying.txt. Wir archivieren ihn mit dem exakten Quellstand und führen ihn für unsere eigenen Beiträge ausdrücklich fort. [S2]

Sanmill nennt fremde Ursprungsprojekte, darunter Stockfish und PerfectAI. Ein Maintainer kann fremde Rechte nicht beliebig erweitern. Deshalb prüfen wir, welche fremden Teile im tatsächlichen iOS-Binary verbleiben und ob ihre Rechte die Store-Verbreitung abdecken. Das ist ein **Release-Kriterium**, keine Behauptung eines bereits festgestellten Verstoßes. Apple verlangt die Einhaltung aller OSS-Lizenzen. [S6, S18]

Layout, Regeln und eigener Name

Ideen und Grundsätze eines Programms sind nach § 69a Abs. 2 UrhG nicht als Programmcode geschützt. Daraus folgt keine freie Nutzung konkreter Grafiken, Texte oder einer individuell gestalteten Oberfläche. Wir übernehmen die funktionale Ordnung und entwickeln eigene visuelle Assets. Code-Portierungen bleiben abgeleitete Bearbeitungen; ein echter unabhängiger Nachbau wäre ein anderer, aufwendigerer Weg. [S19, S20]

Eigener Name und eigene Bundle-ID erfüllen die Vorgabe für inoffizielle Ableitungen. Markenrechte werden durch die AGPL nicht allgemein eingeräumt. Apples Copycat-Regel wird zusätzlich durch sichtbaren gestalterischen und funktionalen Mehrwert adressiert. [S2, S21]

Rechtliche Einordnung

Quellenbasierte Vorprüfung mit Deutschland/EU als Ausgangspunkt. Eine verbindliche Freigabe der Rechtekette und der App-Store-Ausnahme sollte vor dem Vertrieb fachanwaltlich erfolgen. Aktuell ist der Weg tragfähig planbar, aber die Freigabe sämtlicher Fremdteile noch nicht nachgewiesen.

Übernehmen, portieren oder neu erstellen

Bestandteil / Befund	Konsequenz für unser Produkt
Regeln und KI-Suche tgf-core, tgf-mill, tgf-search: AGPL-3.0-or-later. [S3]	Übernehmen und auf einen festen Commit begrenzen. Urheberhinweise erhalten; nur notwendige Anpassungen. Alle späteren Änderungen offen dokumentieren.
Flutter-Oberfläche Dart-Widgets, Dialoge, Animationen, Bedienabläufe. [S7]	Oberfläche in SwiftUI neu schreiben. Erprobte Abläufe und Informationsordnung als Vorlage verwenden. Kein rechtlich notwendiger Komplett-Neubau der KI.
Zugwahl außerhalb der Suche Eröffnungsbuch und Human-DB-Auswahl enthalten Dart-Logik. [S8, S9]	Auswahlregeln gezielt nach Rust portieren oder in einem klar getesteten Swift-Service nachbilden. Bevorzugt Rust für spätere gemeinsame Servernutzung.
Swift/Rust-Brücke Der vorhandene Adapter ist für Flutter Rust Bridge. [S5]	Kleine neue C-ABI mit Swift-Wrapper und XCFramework. Spielsession-Logik aus dem FRB-Paket bei Bedarf in eine unabhängige Rust-Schicht lösen.
Perfect Database Rust-Lader vorhanden; C++-Oracle optional. [S10]	Code und eigentliche Daten getrennt prüfen. Inhalt, Herkunft, Nutzungsrechte, Variantenabdeckung, Größe und iOS-Speicherbedarf entscheiden über das Ausliefern.
Human DB / Eröffnungen / Rätsel Human DB ist extern und wird nicht mitgeliefert. [S8, S9, S11]	Importfunktion möglich. Keine pauschale Freigabe aller fremden Datensätze annehmen. Eröffnungsquellen und Rätselsammlungen einzeln freigeben oder eigene Inhalte erzeugen.
Audio und Bilder Separate Audio-Lizenzen: CC BY-SA 4.0 und Adobe-Audition-Bedingungen. [S12]	Eigene Sounds, Brettgrafik, Steine und Icon erstellen. Bei Übernahme: genaue Dateizuordnung und gesonderte Bedingungen prüfen. AGPL-Store-Ausnahme deckt diese Lizenzen nicht automatisch.
Abhängigkeiten und Herkunft Cargo.lock, Drittbibliotheken und ältere Fremdbeiträge. [S6]	Für das tatsächlich ausgelieferte Paket eine Komponentenliste mit Version, Lizenz, Herkunft und Store-Erlaubnis erstellen. Flutter-Pakete nicht unnötig übernehmen.

Wichtig: CC BY-SA erlaubt kommerzielle Nutzung, fordert aber unter anderem Namensnennung, ShareAlike bei Bearbeitung und den Verzicht auf zusätzliche Einschränkungen. Die Adobe-Datei verweist zusätzlich auf eine EULA. Beide Befunde sprechen für eigene Sounds als einfachsten Weg. [S12, S22]

Quellen: [S3 Rust-Pakete](#) · [S6 Herkunft](#) · [S12 Audio-Lizenzen](#)

Funktionsumfang und Prioritäten

Bestand anhand des Quellcodes und der Dokumentation, keine Zusage über die Fehlerfreiheit der bestehenden App. Der neue Funktionskatalog wird nach der Gestaltung als Abnahmeliste geführt. [S7-S14]

Bereich	Offline-Version 1.0	Ergänzung / Grenze
Spiel und Regeln	Mensch gegen KI, zwei Menschen an einem Gerät, KI gegen KI; Setzen, Ziehen, Schlagen, Fliegen; Sieg und Remis; alle freigegebenen Varianten.	Experimentelle Regeln getrennt kennzeichnen und nur nach eigenen Tests freigeben.
KI-Stufen	Die bestehenden 30 Stufen und ihre Parametrisierung erhalten; verständliche Gruppen in der Bedienung.	Stärke und Rechenzeit auf Zielgeräten messen, keine unbelegte Elo-Werbung.
Analyse und Lernen	Hinweiszug, Stellungsbewertung, Zugliste, Zurück/Vor, Partieanalyse; Stellungen über validierten Import bearbeiten.	Rätselmodus mit eigener oder freigegebener Sammlung; weitere Trainingsformen anschließend.
Partien und Statistik	Automatisches Fortsetzen, lokale Partienbibliothek, Gewinn/Remis/Verlust nach Spielmodus und KI-Stufe; PGN/FEN-Import und Export.	Sanmills Mühle-Dialekt und Regelinformationen explizit erhalten; kein beliebiges Schach-PGN versprechen.
Darstellung und Zugang	Eigene Brettdarstellung, helle/dunkle Variante, Haptik und optionale Sounds; VoiceOver, große Schrift, Kontraste; Deutsch/Englisch.	60+ bestehende Sprachen sind kein automatischer Lieferumfang. Weitere Übersetzungen später separat prüfen.
Eröffnungen und Datenbanken	Eröffnungsbuch bei geklärter Herkunft; technische Schnittstellen für Perfect DB und Human DB.	Datenbankinhalte nur nach Rechte- und Größentest. Ohne sie keine vollständige Zugwahl-Gleichheit versprechen.
Netzwerk	Versionierte Zustände und Zugprotokolle intern vorbereiten.	Online, LAN und Nahbereich bewusst als nachfolgendes Arbeitspaket.

Nachgewiesene Regelvorgaben

Elf Presets: Nine Men’s Morris, Twelve Men’s Morris, Dooz, Morabaraba, Russian Mill, Lasker Morris, Cheng San Qi, Da San Qi, Zhi Qi, El Filja und Experimental. Die IDs 0-10 sind gespeicherte Kompatibilitätsdaten. Hausregeln umfassen unter anderem Fliegen, Mehrfachschiagen, Schlagen aus Mühlen, Bewegen während des Setzens und Wiederholungsremis. [S13]

Der ältere Modus „setupPosition“ ist im aktuellen Code nur noch ein Kompatibilitätswert. Wir planen eine zeitgemäße Stellungsbearbeitung im Analysebereich. Die optionale generative Textanalyse ist ein anderes Feature als die lokale Spiel-KI und gehört nicht zum geplanten Startumfang. [S2, S14]

Layout erhalten, Bedienung modernisieren

Die sichtbare Referenz und der aktuelle UI-Code zeigen das Spielbrett als Schwerpunkt, Spieler- und Zugstatus darüber sowie Verlauf und Aktionen darunter. Diese funktionale Hierarchie bleibt erhalten. Die im Repository enthaltenen Store-Screenshots sind Anschauungsmaterial und kein Nachweis des aktuellen iOS-Laufzeitverhaltens. [S7]

Element der Vorlage	Native Weiterentwicklung
Großes quadratisches Brett	Brett aus den Koordinaten und Verbindungen des Spielkerns zeichnen. Skalierbare Vektoren, präzise Trefferflächen, dezente Animationen.
Spieler- und Zuganzeige	Kompakte Statusbereiche für aktive Seite, Reservesteine, geschlagene Steine und gegebenenfalls Uhr. Status zusätzlich durch Text und Form vermitteln.
Verlaufsnavigation	Einfach zugängliches Zurück/Vor; vollständige Zugliste als ausziehbarer Bereich, auf iPad neben dem Brett. Rücknahme im lokalen Spiel klar von einer Online-Anfrage unterscheiden.
Viele Einstellungen und Aktionen	Häufiges direkt am Brett; seltene Einstellungen in nativen Sheets. Neue Partie in wenigen verständlichen Schritten: Modus, Variante, Seite, Stärke.
Vorhandene Spiel- und Lernbereiche	Vorgeschlagene Bereiche: Spielen, Lernen, Partien und Einstellungen. Sichtbare Hauptnavigation in der Designphase testen.
Themes und Dekoration	Eigenes, ruhiges Erscheinungsbild mit zwei Grundvarianten. Lesbarkeit und eindeutiger Spielzustand haben Vorrang vor Materialeffekten.

Anforderungen an das Grunddesign im nächsten Schritt

Bedienbarkeit: Antippen als vollständiger Bedienweg; Ziehen optional. Auswahl, erlaubte Ziele, Mühlenbildung und erforderliches Schlagen müssen unterscheidbar sein. Keine Information ausschließlich über Farbe, Ton oder Animation vermitteln.

Barrierefreiheit: Das gezeichnete Brett erhält fokussierbare Felder und verständliche VoiceOver-Aktionen. Große Schrift vergrößert Status und Navigation, ohne das Brett abzuschneiden. Bewegungsreduktion und hoher Kontrast werden von Anfang an berücksichtigt.

Geräte: iPhone als Schwerpunkt, adaptives iPad-Layout empfohlen. Kleine Displays, Hoch-/Querformat und Unterbrechungen durch den App-Lebenszyklus gehören in die Entwürfe. Die Mindestversion von iOS wird nach Geräteabdeckung und Prototyp festgelegt.

Gestaltung ist hier noch nicht entschieden

Farben, Materialien, Typografie, Icon, Name und konkrete Screens werden erst im nächsten Schritt erarbeitet. Die Gestaltung dieses Planungsdokuments ist keine Festlegung des App-Designs.

Technische Zielarchitektur

Schicht	Verantwortung und Herkunft
SwiftUI-Oberfläche neu	Brett, Navigation, Eingaben, Animationen, VoiceOver, Einstellungen und Systemdialoge. Das UI stellt ausschließlich den bestätigten Spielzustand dar.
Swift-App-Schicht neu	Partieverwaltung, Autosave, Bibliothek, Statistik, Dateien, Haptik, Lebenszyklus. Ein Engine-Service serialisiert Zugriffe; rechenintensive Suche läuft außerhalb des Main Actor.
Swift/Rust-Grenze neu	Versionierte C-ABI mit undurchsichtigen Session-Handles, expliziter Speicherfreigabe, Fehlercodes und Abbruch. Bibliothek als XCFramework für Gerät und Simulator.
Rust-Spielsession und Zugwahl übernehmen / entkoppeln	Zustand, Zugprüfung, Historie, Suche und Datenquellen. Verwendbare Logik aus tgf-frb herauslösen, ohne generierten FRB-Code nach Swift zu kopieren.
Rust-Kern übernehmen	tgf-core + tgf-mill + tgf-search; optional perfect-db. Eine einzige maßgebliche Regelimplementierung für lokale Spiele und spätere Serverprüfung.
Netzwerkadapter später	Gleicher Session-Vertrag für lokalen Mitspieler, KI und entfernten Spieler. Transport steuert keine Spielregeln.

Kleine, prüfbare Integrationsgrenze

Vorgesehene Operationen: Session anlegen/freigeben, Regelkonfiguration setzen, Topologie und Zustand lesen, legale Aktionen abfragen, Aktion anwenden, zurück/vor, Position laden, Suche starten/abbrechen und Ergebnisse abholen. Eine Revision pro Zustand verhindert, dass ein verspäteter KI-Zug nach einer Rücknahme oder neuen Partie angewendet wird.

Rust kann für ARM64-iOS und den ARM64-Simulator gebaut werden. Das ist eine tragfähige Grundlage, aber noch kein erfolgreicher Build dieses konkreten Projekts. Im Prototyp prüfen wir beide Zielplattformen, Speicherbesitz, Threads, FFI-Fehler und Lebenszyklus auf einem realen iPhone. [S17]

Zustandsmodell für Speichern und späteres Netzwerk

Speichern: Schema-Version, Spiel-ID, Regelversion und vollständige Regeln, Startstellung, Zug-/Aktionsprotokoll, aktive Seite, Phase, ausstehende Schlagaktionen, Remiszähler und relevante Wiederholungshistorie. Snapshots beschleunigen das Laden; die vollständige Partie muss reproduzierbar bleiben.

Das interne 256-Byte-Snapshot-Payload und eine Rust-Speicherrepräsentation sind kein dauerhaftes Dateiformat. Wir definieren ein explizites, versioniertes Austauschformat. FEN allein reicht nicht zuverlässig für alle historischen Remisbedingungen. [S4]

Quellen: [S4 Framework-Vertrag](#) · [S5 Flutter-Adapter](#) · [S17 Rust auf iOS](#)

KI und Regelgleichheit

Was wir tatsächlich übernehmen

Der Rust-Suchkern enthält Alpha-Beta/PVS, MTD(f), iterative Vertiefung, MCTS, Transpositionstabellen, Zugordnung und Abbruchmechanismen. Bewertungslogik, Schwierigkeitsparameter und Spielregeln liegen in getrennten Komponenten. Wir erhalten diese zunächst möglichst unverändert. [S3, S4]

Gleicher Suchkern bedeutet noch nicht gleiche Zugwahl

Sanmill konsultiert optional zuerst das Eröffnungsbuch, dann die Human Database und anschließend die native Suche. Die Human-DB-Auswahl kann durch eine Perfect Database korrigiert werden. Auswahlstrategien und Zufallseinstellungen sind Teil der Produktlogik. Sie müssen gemeinsam mit dem Kern übertragen und getestet werden. [S8, S9]

Abnahme für Regeln und Portierung

Prüfung	Erwartetes Ergebnis
Regelgleichheit	Für jedes freigegebene Preset: gleiche legalen Aktionen, Folgezustände, Schlagpflichten, Phasenwechsel und Ergebnisse wie am eingefrorenen Referenzstand. Randfälle und ungültige Eingaben enthalten.
Deterministische Suche	Gleiche Position, Historie, Optionen, feste Tiefe und Seed; zunächst ein Thread. Zug und Bewertung stimmen überein. Bei unveränderter Suchlogik zusätzlich Knotenanzahl vergleichen.
Praktische Spielstärke	Bei Zeitlimit oder mehreren Threads sind abweichende Züge möglich. Paarweise Partien mit wechselnden Seiten und identischen Zeitbudgets; Ergebnisse mit Unsicherheit bewerten.
Datenquellen	Feste Fixtures für Buch, Human DB und Perfect DB; Varianten, Symmetrien, fehlende Dateien und fehlerhafte Daten. Auswahlreihenfolge und Rückfall auf Suche explizit prüfen.
Fortsetzen und Import	Speichern/Laden sowie PGN-/FEN-Roundtrips erzeugen identische zulässige Zustände. Remis-Historie und mehrteilige Züge bleiben erhalten.
iOS-Integration	Keine blockierende Suche auf dem UI-Thread, keine veralteten Resultate, keine Speicherfehler an der Brücke. Pause, App-Wechsel, Sperrbildschirm und Speicherknappheit prüfen.

Messziele für den Prototyp

Auf einem festgelegten älteren unterstützten iPhone: unmittelbares visuelles Eingabefeedback (Ziel unter 100 ms), flüssige Brettbedienung während der Suche und zügiger Suchabbruch (Ziel unter 250 ms). Suchzeit, Energiebedarf und Spitzen-RAM pro Stufe messen. Diese Werte sind geplante Akzeptanzziele, noch keine Messergebnisse.

Vorhandene Rust-Tests und historische Referenzstellungen werden übernommen. Ihre Existenz ersetzt weder einen eigenen Testlauf noch die Prüfung der neuen Swift-Anbindung. In dieser Vorplanung wurden keine App-Builds oder Engine-Tests ausgeführt.

Arbeitspakete und Meilensteine

Schätzung in Personentagen (PT), jeweils fünf Arbeitstage pro Woche. Annahme: erfahrene iOS-Entwicklung mit Rust-Unterstützung, kleine eigenständige Gestaltung und fokussierte Qualitätssicherung. Zeiten sind Planwerte, keine Lieferzusage.

Phase	Ergebnis und Abhängigkeit	PT
0 · Rechtebasis	Rechteverzeichnis für Kern, Fremdbeiträge und geplante Daten; App-Store-Ausnahme bewerten. Essential-Code klären, optionale ungeklärte Assets ersetzen oder zurückstellen. Parallel zur Gestaltung möglich.	3-5
1 · Name und Grunddesign Nächster Schritt	Namensauswahl, Kollisionsprüfung, zwei Gestaltungsrichtungen, vier zentrale Screens, Bedienabläufe. Abschluss: Name und Designrichtung gemeinsam festlegen.	5-8
2 · Projekt + Prototyp	Erst danach richtig benannten Projektordner, Repository und Xcode-Projekt anlegen. Rust-Bibliothek für Gerät/Simulator anbinden; vollständige Testpartie, Suchabbruch und Referenzvergleich.	5-8
3 · Spielbare Grundlage	Regeln/Varianten, menschliche Eingabe, KI, vollständige Züge, Partieende, lokale Zweispielerpartie, Autosave. Abschluss: belastbare interne Alpha.	10-15
4 · Native Produktqualität	Adaptives UI, Barrierefreiheit, Einstellungen, eigene Assets, Partienbibliothek, Wiederaufnahme, Deutsch/Englisch. Abhängig von Design und stabiler Session.	12-18
5 · Funktionsgleichwertigkeit	30 KI-Stufen, Zugwahlquellen, Analyse, Verlauf, Import/Export, Statistik und fokussierter Rätselmodus. Daten nur mit Freigabe. Abnahme gegen die Funktionsmatrix.	15-25
6 · Beta und Veröffentlichung	Geräte-/Regeltests, TestFlight, Fehlerbehebung, Quellenpaket, Lizenzanzeige, Privacy-Angaben, Support, Store-Material und 0,99-€-Preiseinstellung.	8-12

Gesamt und kritischer Pfad

58-91 PT plus 20 % Reserve = rund 70-110 PT. Der Ablauf hängt vor allem an Rechtebasis, Name/Design, erfolgreicher Engine-Brücke und überprüfter Regel-/KI-Gleichwertigkeit. Bei einer Person seriell etwa 14-22 Arbeitswochen; bei mehreren Beteiligten nicht linear teilbar.

Nicht enthalten: Aufbau großer neuer Datenbanken, umfassende Freigabe fremder Sammlungen, 60+ Übersetzungen, generative Textanalyse, laufender Support und Netzwerkbetrieb. Falls Datenbanken nicht rechtzeitig geklärt sind, bleibt das Produkt mit Such-KI möglich; deren Auslassung muss als bewusste Umfangsentscheidung bestätigt werden.

Meilenstein vor größerem Implementierungsaufwand

Nach Phase 2 muss eine Partie auf einem realen iPhone mit derselben Regel- und Suchbasis funktionieren. Erst dann werden Aufwand, Mindest-iOS und Datenbankumfang verbindlicher geplant.

Netzwerk als zweite Produktstufe

Sanmill enthält bereits LAN-/Bluetooth-Transporte, einen Cloud-Match-Koordinator und einen Online-Worker mit Regelprüfung. Wir prüfen diese als Referenz; es besteht keine automatische Berechtigung, Sanmills produktive Server für unsere App mitzubenutzen. Die tatsächliche Betriebsqualität dieser Komponenten wurde hier nicht getestet. [S15]

Option	Passung und Aufwand
A · Game Center / GameKit Empfehlung für den ersten iOS-Ausbau	Einladungen und rundenbasierte Partien mit Apple-Infrastruktur. Für Mühle passen asynchrone Züge gut. Weniger eigene Betriebsverantwortung; an Apples Plattform und Konten gebunden. Grob 15-25 PT + Reserve, etwa 4-7 Wochen. [S23]
B · Eigener Online-Dienst Bei plattformübergreifendem Ziel	Eigene Lobby/Einladung und serverseitige Regelvalidierung, gegebenenfalls aus Sanmills Worker abgeleitet. Mehr Kontrolle über Synchronisierung und Ranglisten; Hosting, Sicherheit, Datenschutz und Betrieb nötig. Grob 30-55 PT + Reserve, etwa 8-14 Wochen.
C · Spiel in der Nähe Separates optionales Paket	Lokales Netz oder Apple-Nahbereichsanbindung nach Geräteprüfung. Kein Ersatz für Internet-Multiplayer. Kompatibilität zum vorhandenen Sanmill-Protokoll nur nach eigenem Interoperabilitätstest zusagen. Aufwand zusätzlich planen.

Was wir dafür schon in Version 1 vorbereiten

Stabile Spiel-/Regel- und Protokollversionen; eindeutige Match- und Command-IDs; Sequenznummern, Zustandsrevision und Prüfsummen; klarer Session-Vertrag. Ein kompletter logischer Zug kann aus Setzen/Ziehen und mehreren Schlagaktionen bestehen. Der Zugwechsel erfolgt erst, wenn alle Pflichtaktionen abgeschlossen sind.

Abnahmekriterien für Online-Spiele

Einladung, Beitritt und Regelabgleich funktionieren auf zwei Geräten. Doppelte, verspätete oder vertauschte Nachrichten verändern eine Partie nicht doppelt. Nach Verbindungsabbruch wird aus bestätigtem Zustand und Protokoll wiederhergestellt. Aufgabe, Remis, Zeitüberschreitung und Rücknahme besitzen eindeutige Regeln.

Bei GameKit prüfen beide Clients alle empfangenen Zustände; GameKit kennt unsere Mühle-Regeln nicht. Für belastbare gewertete Spiele bevorzugen wir serverseitige Autorität. Offener oder geschlossener Quellcode verhindert die Nutzung fremder KI-Hilfen durch Spieler nicht.

Lizenz, Preis und Betrieb

Ein veränderter AGPL-Dienst muss den betroffenen Netzwerk-Nutzern den zugehörigen Quellcode anbieten. Ein getrennter Dienst macht nicht automatisch jeden anderen Serverbestandteil AGPL-pflichtig; die Kopplung ist zu bewerten. Einmalig 0,99 € finanziert keine unbegrenzten Betriebskosten: vor einem eigenen Dienst Kosten, Aufbewahrung und Support festlegen. [S1]

Risiken und nächste Schritte

Offener Punkt	Konkrete Erledigung / Verantwortung
Reichweite der Store-Ausnahme	Entwicklung erstellt die Herkunftsliste der tatsächlich eingebundenen Dateien; Rechtsberatung bewertet Fremdbeiträge und Ausnahme. Ungeklärte essentielle Teile vor Integration entscheiden.
Datenbank- und Asset-Rechte	Entwicklung/Produkt ordnen jede übernommene Datei einer Quelle und Lizenz zu. Datenbankrechte, etwaige Rechte an Sammlungen und Store-Bedingungen gesondert berücksichtigen.
Technischer Reifegrad	Entwicklung führt im Prototyp iOS-Builds, Session- und Suchtests aus. Der geprüfte Stand ist ein Entwicklungsstand, keine hier nachgewiesene stabile Release-Basis.
Umfang und Aufwand	Produkt priorisiert Analyse/Rätsel/Datenbanken nach dem Grunddesign. Alle Abweichungen von der geplanten Funktionsmatrix werden ausdrücklich festgehalten.
Vertrieb für 0,99 €	Produkt richtet später Paid Apps Agreement, Steuer-/Bankdaten und regionale Preise ein. 0,99 € ist Ziel-Endkundenpreis in Deutschland, nicht Nettoerlös und nicht automatisch Preis in jedem Land. [S24]

Nächste Sitzung: Namensgebung und Grunddesign

1. Positionierung und Namen: Ein ruhiges, hochwertiges Mühlespiel mit starker Offline-KI. Mehrere unterscheidbare Namensrichtungen erarbeiten; danach 3-5 Kandidaten prüfen. App Store, Web, Domains und Markenregister (DPMA, EUIPO/TMview, gegebenenfalls WIPO) berücksichtigen. Ein freier Domainname ersetzt keine Markenprüfung. [S25]

2. Zwei Gestaltungsrichtungen: Beispielsweise reduziert-geometrisch und warm-materialbetont als Diskussionsgrundlage. Pro Richtung: Brett, Steine, Status, Navigation, Schrift, Hell/Dunkel und Icon-Idee. Noch keine Richtung beschlossen.

3. Vier zentrale Screens: Start/Partiefortsetzung, laufende Partie, neue Partie/Regeln und Analyse/Verlauf. Mindestens ein kleines iPhone sowie ein adaptives iPad-Beispiel; VoiceOver- und Großschriftkonzept einbeziehen.

4. Gemeinsame Festlegung: Name, sichtbarer App-Titel, technische Schreibweise, Bundle-ID unter eigener Domain und Designrichtung beschließen. Erst danach Projektordner und Xcode-Projekt erstellen. Die Dokumentgestaltung hier bleibt davon unabhängig.

Vor dem Store-Upload

Veröffentlichtes Quellarchiv passend zum Binary, übernommene Lizenzhinweise und eigene Änderungshinweise; Rechtstexte und Quelle in der App erreichbar. Datenschutzangaben passend zu den tatsächlich verwendeten SDKs, Support-Kontakt, Altersfreigabe, Geräteprüfung und eigenständige Store-Screenshots. App-Review-Notizen erklären Herkunft und native Weiterentwicklung. [S18, S21, S24]

Aktueller Stand: Recherche und Planung. Noch kein benanntes Produktprojekt, keine App-Implementierung und keine Veröffentlichung.

Quellen und Prüfgrundlage

Geprüfter Repository-Stand: **8901a06f088b**

Commit vom 15.09.2026, 21:31:20 +0800; abgerufen am 30.09.2026. Die lokalen Quellen sind auf diesen Commit begrenzt. Webseiten und Store-Regeln wurden zum Planungsdatum recherchiert.

- S1** · AGPL-Lizenztext; insbesondere Abschnitte 1, 4-7, 10, 13
- S2** · README: App-Store-Zusatzerlaubnis, Fork-Kennzeichnung, optionale Textanalyse
- S3** · Rust-Pakete: Lizenz und Abhängigkeiten (tgf-search; ebenso core und mill)
- S4** · TGF Framework API: Regeln, Topologie, Sucher und Zustandsgrenzen
- S5** · Flutter-Rust-Adapter: Paket und bestehende iOS-staticlib
- S6** · AUTHORS: benannte Fremdprojekte und Urheber
- S7** · Spielseite; zusätzlich Spielbrett-Code und Store-Screenshot 1 geprüft
- S8** · Eröffnungsbuch: Datenquellen und Reihenfolge der Zugwahl
- S9** · Human Database: externe SQLite-Datei und Auswahlstrategie
- S10** · Perfect DB: Rust-Lader und optionales C++-Oracle
- S11** · Rätsel-Import und Export
- S12** · Separate Audio-Lizenzdateien: Matt Littlewood und Adobe Audition
- S13** · Elf Presets; Regeloptionen ergänzend in rules/types.rs
- S14** · Spielmodi und stillgelegter Setup-Kompatibilitätswert
- S15** · Vorhandener Online-Worker; ergänzend remote_play und online_play geprüft
- S16** · Funktionsbeschreibung des Projekts
- S17** · Rust: offiziell unterstützte iOS-Targets
- S18** · Apple Developer Program License Agreement, insbesondere FOSS-Klausel
- S19** · § 69a UrhG: Ausdrucksformen, Ideen und Grundsätze
- S20** · § 69c UrhG: zustimmungsbedürftige Handlungen und Bearbeitungen
- S21** · Apple App Review Guidelines, insbesondere 4.1, 5.1 und 5.2
- S22** · Creative Commons: CC BY-SA 4.0, Zusammenfassung und Lizenzverweis
- S23** · Apple GameKit: rundenbasierte Spiele
- S24** · Apple: App-Preise festlegen, Vertrag und Länderpreise
- S25** · DPMA: Markenrecherche und Grenzen der Registersuche

Prüfgrenzen: Statische Auswahlprüfung von Quellcode, Konfigurationen, Lizenztexten, Audio-Lizenzdateien und einem Referenz-Screenshot. Kein vollständiges transitive-Abhängigkeiten-Audit, keine lückenlose Rechtekettenprüfung, keine Markenrecherche zu einem noch ungewählten Namen, kein App-Build, keine Laufzeit-, Stärke- oder Netzwerkprüfung. Empfehlungen und Schätzungen sind als Planung zu verstehen.